

LAB : 5

Title: the while loop in c++

1. Objective

The main goal of this lab is to practice using different types of loops in C++. Loops are one of the most important concepts in programming because they allow us to repeat tasks without writing the same code again and again. In this lab, we focus on:

- The while loop
- The nested while loop
- The do...while loop

By the end of this lab, students should be able to:

- Write programs that use loops to display sequences of numbers.
- Understand the difference between while and do...while loops.
- Apply loops to solve problems like summation.
- Recognize how nested loops can be used to print patterns.

2. Theory

2.1 While Loop

The while loop is a control structure that repeats a block of code as long as a condition is true. The condition is checked before the loop body runs. If the condition is false at the start, the loop body will not run at all.

General Syntax:

```
initialization;  
  
while (condition) {  
    statements;  
    update variable;  
}
```

Key Points:

- Initialization sets the starting value.
- The condition decides whether the loop continues.
- The update changes the variable so the loop eventually stops.

2.2 Nested While Loop

A nested loop means one loop inside another. The outer loop controls the number of rows, and the inner loop controls the number of columns or repeated values. This is useful for printing tables, grids, or number patterns.

2.3 Do...While Loop

The do...while loop is similar to the while loop but with one difference: the body runs at least once before the condition is checked. This is useful when we want the program to ask for input or perform an action at least once, even if the condition is false later.

General Syntax:

```
initialization;  
  
do {  
    statements;
```

```
    update variable;  
} while (condition);
```

3. Programs and Outputs

Task 1: Print numbers 1 to 10 using while loop

```
#include <iostream>    // tells the compiler to include input output streamlibrary  
  
using namespace std;    // allows us to use cout without typing std:: for each  
  
int main() {            // the point from where function execution begins  
  
    int i = 1;           // variable declaration and assigning value of 1  
  
    while (i <= 10) {    // while condition in which the loop will run if i is <or= to 10  
  
        cout << i << endl;    // sends result to output  
  
        i++;    }          // adds 1 to the i until condition becomes false  
  
  
    return 0;            // tells the computer that program has ended successfully  
}
```

Output:

1,2,3,4,5,6,7,8,9,10 in a column

Task 2: Repeat Task 1 using do...while loop

```
#include <iostream>    // tells the compiler to include input output stream library  
  
using namespace std;    // allows us to use cout without typing std:: for each  
  
int main() {            // allows us to use cout without typing std:: for each  
  
    int i = 1;           // variable declaration and assigning value of 1
```



```

do {                                // prints code in the body first once
    cout << i << endl;              // output the value of i to user
    i++;                            // adds one to the output every time
} while (i <= 10);                  // while condition in which condition is given
return 0;                          // tells the computer that program has ended successfully
}

```

Output:

1 2 3 4 5 6 7 8 9 10 in a column

Task 3: Sum of numbers from 1 to n using while loop

```

#include <iostream> // tells the compiler to include input output stream library
using namespace std; // allows us to use cout without typing std:: for each
int main() {        // allows us to use cout without typing std:: for each
    int n, sum = 0, i = 1; // variable declaration and assigning values
    cout << "Enter a positive integer: "; //prompt for user to input integer
    cin >> n;        // takes input from user and stores it in variable n

    while (i <= n) { // while condition which runs until i<=n
        sum += i;    // addition formula used to add numbers
        i++;         // adds one to the sum every time
    }                // closing body

    cout << "Sum = " << sum << endl; // outputs sum to the user
    return 0;                      // tells the computer that program has ended successfully
}

```

Output:

Enter a positive integer: 5

Sum = 15

4. Discussion

This lab showed how loops make programming easier and more efficient. Without loops, we would have to write many lines of code to print numbers or calculate sums. With loops, the same task can be done in just a few lines.

- The while loop is best when we don't know how many times the loop will run, but we have a condition to check.
- The do...while loop is useful when we want the loop to run at least once, such as asking for a password or user input.
- The nested while loop allows us to create patterns and handle multiple levels of repetition.
- By practicing these loops, I learned how to control the flow of a program and how to avoid infinite loops by updating the loop variable correctly.

5. Conclusion

In this lab, I successfully wrote programs using while loops, do...while loops, and nested loops. I learned the differences between them and how each one is applied in real problems.

- The while loop checks the condition first.
- The do...while loop runs at least once before checking the condition.
- Nested loops are powerful for printing patterns and handling multi-level tasks.

This lab improved my confidence in using loops and gave me practical experience in writing programs that repeat tasks automatically. Loops are a fundamental concept in programming, and mastering them will help me in future labs and projects.

